

## Task: Corporate HR Management System

### Objective

Build a modular HR management system for a corporate company that can:

- Manage employee profiles, departments, job roles, and payroll records
- Use Pydantic for input validation
- Use SQLAlchemy ORM for database models and queries
- Use Typing for all function definitions
- Use Logging for tracking HR operations

You are free to design the relationships, models, and schema structures as you think best — but make sure the system is modular and realistic.

### Tables

- Department
- Employee
- Payroll
- LeaveRecord

Create appropriate Pydantic schemas for each table.

Decide what fields and relationships each table should have (e.g., department-employee mapping, payroll linkage, etc.).

### Requirements

- Implement basic CRUD for each entity (create, read, update, delete)
- Include validation and data consistency checks
- Write clean, reusable code with proper function definitions and annotations
- Organize your code into separate files (models.py, schemas.py, etc.)

### Logging Requirements

Create a separate logger\_config.py file to configure logging.

- Log to both console and file (hr\_app.log)
- Use the following levels:
  - INFO: e.g., new employee added, payroll generated, leave approved
  - WARNING: e.g., department over budget, inactive employee payroll attempt
  - ERROR: e.g., invalid input, DB integrity issues

Make sure logs are descriptive and timestamped.

## Typing Requirements

Use type hints consistently across all modules.

- Use types such as `List[Employee]`, `Optional[Department]`, `Dict[str, float]`, etc.
- Annotate every function and class method.

Example:

```
def get_active_employees() -> List[Employee]:  
    ...
```

## Deliverables

- Source code with modular structure
- Properly configured logger
- Example test script (`main.py`) showing creation and interaction of records
- SQLAlchemy database (SQLite recommended)